

AI-Based Resume Screening System

Project Report on an NLP and Machine Learning based candidate ranking system

Project Type	Artificial Intelligence / Machine Learning
Frontend	Streamlit
Core Language	Python
Main Objective	Rank resumes according to job description relevance

Abstract

The AI-Based Resume Screening System is an intelligent application that automates the initial stage of recruitment by analyzing resumes and matching them against a job description. The system uses Natural Language Processing to parse and preprocess resume content, Machine Learning techniques to represent text numerically, and cosine similarity to calculate how closely each resume matches the job requirements. The final output is a ranked list of candidates with match percentage, extracted skills, missing skills, and shortlist status.

Introduction

Manual resume screening is time-consuming and can become inconsistent when recruiters evaluate a large number of applications. This project reduces manual effort by providing a structured and repeatable screening workflow. The application supports multiple resume uploads, accepts a job description, analyzes candidate profiles, and presents ranked results through a web interface.

Objectives

- Parse resumes from PDF, DOC, DOCX, and TXT formats.
- Preprocess resume and job description text for better matching.
- Extract relevant technical skills from resumes.
- Generate text vectors or semantic embeddings.
- Calculate similarity between resumes and job descriptions.
- Rank candidates according to match score.
- Display results using a user-friendly Streamlit interface.

System Architecture



Component-Level Architecture

Component	Description
Input Layer	Accepts multiple resume files and a job description text input.
Document Parsing Layer	Extracts plain text from PDF, DOCX, DOC, and TXT files.
NLP Preprocessing Layer	Cleans text using lowercasing, special character removal, stop word removal, lemmatization fallback, and skill normalization.
Feature Representation Layer	Creates TF-IDF vectors or Sentence Transformer embeddings.
Similarity Computation	Uses cosine similarity to compare each resume with the job description.
Ranking Engine	Sorts resumes by score and applies a shortlist threshold.
Output Layer	Displays rank, match percentage, matched skills, missing skills, and candidate status.

Technology Stack

Layer	Technology Used
Programming Language	Python
Frontend	Streamlit
Data Handling	Pandas, NumPy
Machine Learning	scikit-learn
Text Matching	TF-IDF, cosine similarity, Sentence Transformers
Document Parsing	pdfplumber, PyMuPDF, python-docx, Apache Tika
Optional Explainability	SHAP, LIME, LLM-generated explanations

Models Used

- TF-IDF baseline for quick and lightweight text matching.
- Cosine similarity for resume and job description comparison.
- Sentence Transformers such as all-MiniLM-L6-v2 for advanced semantic matching.
- Optional classifier models such as Logistic Regression, Random Forest, or XGBoost for shortlist/reject classification.

Implementation Modules

File	Purpose
app.py	Main Streamlit application and user interface.
src/parser.py	Extracts resume text from supported document formats.
src/preprocess.py	Cleans and normalizes text before matching.
src/vectorizer.py	Generates TF-IDF vectors or transformer embeddings.
src/matcher.py	Computes cosine similarity scores.
src/ranker.py	Ranks candidates and assigns shortlist status.
src/skill_extractor.py	Extracts matched and missing skills.
utils/helpers.py	Provides helper functions for uploaded files and display formatting.

Application Workflow

- The user uploads one or more resumes through the Streamlit interface.
- The user enters the job description in the input box.
- The parser converts each resume into plain text.
- The preprocessing module cleans resume and job description text.
- The vectorizer converts text into TF-IDF vectors or semantic embeddings.
- The matcher calculates cosine similarity scores.
- The ranker sorts candidates and marks each as Shortlist or Review.
- The UI displays a ranked table and match score visualization.

Example Output

Rank	Candidate	Match Score	Status	Extracted Skills	Missing Skills
1	resume_01.pdf	87%	Shortlist	Python, SQL, Machine Learning	Docker
2	resume_02.pdf	64%	Review	Python, Streamlit	SQL, AWS

Testing

The project includes automated tests for preprocessing and ranking behavior. A manual smoke test was also performed for the TF-IDF ranking pipeline. The sample test confirmed that a resume containing Python, SQL, and machine learning skills ranks higher than an unrelated resume and receives Shortlist status when it passes the threshold.

Test Area	Result
Syntax compilation	Passed
Unit tests	Passed
Dependency imports	Passed for core dependencies
TF-IDF ranking pipeline	Passed
Streamlit web endpoint	Responded successfully

Advantages

- Reduces manual resume screening effort.
- Provides consistent ranking criteria.
- Highlights matched and missing skills.
- Supports both baseline and advanced semantic matching.
- Can be extended with explainability and deployment features.

Limitations

- Accuracy depends on the quality of resume text extraction.
- Skill extraction uses a predefined skill dictionary in the current version.
- Sentence Transformer mode requires additional dependencies and model availability.
- The system should be audited for bias before real-world hiring use.

Future Improvements

- Add recruiter feedback loop to improve ranking quality.
- Improve extracted skills using named entity recognition.
- Generate LLM-based candidate explanations.
- Add bias detection and fairness monitoring.
- Deploy the application online.
- Add candidate comparison and analytics dashboard.

Conclusion

The AI-Based Resume Screening System demonstrates how NLP and Machine Learning can simplify candidate screening. By combining resume parsing, preprocessing, vectorization, cosine similarity, ranking, and a Streamlit interface, the project provides an efficient and practical solution for shortlisting candidates according to job relevance.

References

- scikit-learn documentation for TF-IDF and cosine similarity.
- Sentence Transformers documentation for semantic embeddings.
- Streamlit documentation for web application development.
- pdfplumber, PyMuPDF, and python-docx documentation for document parsing.
- O*NET and ESCO skill taxonomies for future skill database expansion.